10-Day Hands-On Interview Prep Plan

AI for Car R&D: CAD Data Pipeline & Similar-Part Search

Goal: In 10 days, build a small, working version of a CAD-to-AI pipeline on your own laptop, so that in the interview you can say: "*I built this, here's what broke, and here's what I'd change at Hyundai's scale.*"

Principle: Cut scope, not steps. A thin system that works end to end beats a perfect half-system.

---

1. The Project at a Glance

You will build a mini **"similar-part search + Q&A"** system:

1. Read public CAD files (STEP)
2. Extract metadata (volume, bounding box, faces)
3. Convert to meshes and render images
4. Embed images and search for similar parts
5. Run it as a pipeline (queue, worker, validation, failures)
6. Add text Q&A over fake PLM records (RAG with citations)
7. Wrap it in a simple UI
8. Measure it and turn the results into an interview story

How it maps to the real architecture

Real system layer	Your mini version
CAD / PDM vault, PLM (vehicle design reportedly mainly in CATIA)	A folder of STEP files (plus one part you model yourself) + a SQLite table of fake part records
Ingestion & conversion	Folder watcher, simple queue, worker, validation, "failed" folder
Metadata extraction	CadQuery / pythonocc properties
Rendering	pyrender headless, 6 views per part
Geometry ML	DINOv2 or CLIP image embeddings
Storage	Local folders + SQLite + Qdrant (or pgvector)
AI services	Similar-part search + RAG Q&A
Engineer apps	Streamlit app

CAD basics you should be able to explain

- **CAD** is the category (any design software). **CATIA** (Dassault Systèmes) is one CAD product, widely used by car and aircraft makers for complex 3D parts, curved body surfaces and large

assemblies. **AutoCAD** (Autodesk) is mainly 2D drafting, common in factory layout, equipment and utilities work.

- Other 3D CAD tools: Siemens NX, PTC Creo, SolidWorks, Autodesk Fusion. Each saves in its own format.
- **CATIA file types:** `CATPart` (a single part) and `CATProduct` (an assembly that references many CATParts).
- **Feature tree / design intent:** native CAD stores how a part was built (sketches, extrusions, fillets, parameters). **STEP** stores the final exact geometry (B-rep) and, in AP242, PMI (dimensions and tolerances), but usually not the editable feature history.
- **Why STEP in the pipeline:** it is the common language every CAD tool can export, and open-source tools can read it.

> Note: "Hyundai mainly uses CATIA for vehicle design" comes from anonymous mentor answers on Linkareer (2024). Say "my understanding is..." unless the job posting confirms it.

---

2. Data and Tools

Data (public, free)

- **ABC dataset**: about 1 million CAD models in STEP format, downloadable in chunks. One chunk is plenty.
- **Fusion 360 Gallery dataset**: alternative with real-world designs.
- Target: **300 to 2,000 parts**. Start with 100 if your laptop is slow.

Tools (Python)

Purpose	Tool
Read STEP, B-rep properties	CadQuery or pythonocc-core (install via conda)
Meshes, export glTF / STL	trimesh
Headless rendering	pyrender (or VTK / Blender in background mode)
Image embeddings	DINOv2 or CLIP (Hugging Face)
Vector search	Qdrant (Docker or local mode) or pgvector
Metadata DB	SQLite (or Postgres)
Queue	Redis + RQ, or a simple Python queue
UI	Streamlit
LLM for Q&A	Any LLM API, or a local model via Ollama

Setup tips

- Use **conda** for pythonocc / CadQuery. Pip installs of OpenCascade often fail.

- Keep one project folder: `data/raw`, `data/step_ok`, `data/failed`, `data/mesh`, `data/renders`, `db/`, `src/`.
- Use AI tools to write code. Your job is to **run it, debug it, and understand why each part exists.**

---

3. Day-by-Day Plan

Day 1: Read CAD

Do

- Set up the environment (conda, CadQuery or pythonocc).
- Download one dataset chunk.
- Write a script that loads each STEP file and extracts: volume, surface area, bounding box, face count, edge count.
- Save results to SQLite.

Done when: you have a table of properties for 100+ parts, and a list of files that failed to load (and why).

Learn: STEP / B-rep basics, why some files fail, what "valid solid" means.

Day 2: Model One Part Yourself + Convert to Meshes

Do (first 2 to 3 hours): model a part

- In a free CAD tool (FreeCAD, or Onshape's free plan), model a simple bracket: sketch, extrude, add holes and fillets, set one or two dimensions as parameters.
- Change a parameter and watch the model update: this is "design intent."
- Export it to STEP, then reopen the STEP: notice the feature tree is gone and only the geometry remains.
- Run your Day 1 script on it, and include it in every later step (it becomes "your" part in the demo).

Do (rest of the day): convert to meshes

- Tessellate each part into a mesh.
- Export **coarse glTF** (viewing) and **fine STL** (ML).
- Try 2 to 3 mesh tolerances; compare triangle counts and file sizes.
- View a few results in a browser glTF viewer or `trimesh.show()`.

Done when: your own bracket exists as native file + STEP; meshes exist for most parts; and you have a small table of tolerance vs. size vs. visual quality.

Learn: feature tree vs. exact geometry vs. mesh (three levels of information loss); tessellation trade-offs; why tolerance should scale with part size.

Day 3: Render Images

Do

- Render 6 fixed views per part (front, back, left, right, top, isometric), headless.
- Normalize: center the part, scale to fit, same lighting, same background.
- Render a few hundred parts.

Done when: a folder of consistent renders, `renders/{part_id}/view_0..5.png`.

Learn: why consistency matters for similarity search.

Day 4: Similar-Part Search (the demo moment)

Do

- Embed each view with DINOv2 or CLIP, average the views into one vector per part.
- Store vectors in Qdrant with `part_id` as payload.
- Query: given a part, return the 5 most similar parts with their images.
- Manually check 10 queries: are results sensible?

Done when: you can pick any part and see visually similar results.

Learn: embeddings, cosine similarity, where image-based similarity fails (e.g., same shape, different size).

Day 5: Make It a Pipeline

Do

- Folder watcher: new file in `data/raw` creates a job.
- Queue (Redis + RQ or simple Python queue) and a worker that runs: load, extract, mesh, render, embed, store.
- **Hash-based skipping:** if the file hash is unchanged, skip.
- **Validation:** volume > 0, bounding box plausible, triangle count within bounds.
- Failed jobs go to `data/failed` with an error log (your dead-letter queue).
- Record lineage: source hash, tool versions, timestamps.

Done when: dropping a new STEP file into the folder makes it searchable without manual steps.

Learn: idempotency, retries, dead-letter queues, lineage.

Day 6: Text Side (RAG Q&A)

Do

- Generate fake PLM records per part: name, material, program, supplier, status, 1 to 3 short test notes.
- Chunk and embed the text; store in the vector DB.
- Build Q&A: hybrid retrieval (keyword + vector), then an LLM answers **with citations** to part IDs.

- Add a simple access filter: each record has a `program` tag; a "user" can only see their programs.

Done when: you can ask "Which aluminum parts failed durability tests?" and get a cited answer, and a restricted user cannot see other programs' parts.

Learn: RAG, citations, permission filtering before retrieval (not after).

Day 7: UI

Do

- Streamlit app with: search bar, part detail page (renders + metadata), "similar parts" panel, chat box.
- Optional: embed a 3D glTF viewer.

Done when: a 2-minute demo flows smoothly from search to similar parts to Q&A.

Learn: what engineers would actually click; where latency hurts.

Day 8: Measure

Do

- Conversion success rate (% loaded, % meshed, % passed validation)
- Average time per part, per stage
- Storage multiplier: total generated data vs. original STEP size
- Search quality: on 20 test queries, how many top-5 results are relevant?
- Write down every failure and its cause.

Done when: a one-page results table plus a failure list.

Learn: these numbers and failure stories are your strongest interview material.

Day 9: Build the Story

Do

- Draw a one-page architecture diagram (your mini version and the scaled-up version).
- Write "what changes at Hyundai scale":
  - **CATIA specifics:** `CATProduct` assemblies reference many `CATPart` files, so conversion must resolve references and handle missing children; batch conversion needs licensed translators, and seat count limits throughput; export to STEP AP242 keeps geometry and PMI but loses the feature tree and parameters
  - Using your own bracket as the example: "When I exported my part, the design intent was lost. At scale, that's why the pipeline keeps PLM metadata alongside STEP."
  - PLM integration and change events
  - Millions of parts, backfill strategy
  - Security: on-prem models, program-level access control, audit
  - Storage thresholds (~70% alert) and 3-2-1 backups
- Prepare a 3-minute walkthrough and a 10-minute deep dive.

Done when: you can explain the system out loud without notes.

Day 10: Mock Interview + Buffer

Do

- Practice out loud (record yourself, or ask a friend or an AI to interview you).
- Rehearse the demo; prepare screenshots in case live demo fails.
- Use remaining time to fix whatever slipped.

---

4. If You Fall Behind

Situation	Cut this	Keep this
Install problems on Day 1	pythonocc; use CadQuery or pre-meshed data	Loading + properties
Rendering too slow	6 views to 3; fewer parts	Consistent renders
Queue too complex	Redis; use a simple loop with status table	Hash skip, validation, failed folder
RAG behind schedule	Hybrid search; use vector only	Citations, access filter
UI behind	3D viewer	Search + similar parts + chat

---

5. Likely Interview Questions

System design

- Design an AI tool that helps car engineers find and reuse existing parts.
- How would you ingest millions of CAD files from PLM? How do you handle changes?
- How do you keep confidential vehicle program data secure in an AI system?

Technical depth

- Why convert to STEP? What is lost in conversion?
- What's the difference between CATIA and CAD? Between CATIA and AutoCAD?
- How would you handle a `CATProduct` assembly with thousands of referenced parts?
- What is design intent, and can an AI system use it?
- Image embeddings vs. geometric (B-rep / point cloud) embeddings: trade-offs?
- How would you evaluate similar-part search quality?
- Where does access control belong in a RAG pipeline?
- How do you size storage and backups?

Product / impact

- Who are the users, and how many? (R&D population ~10,000+; realistic active users a fraction of that.)

- What would you build first, and why?
- How would you measure success? (Adoption, reused parts, time saved, duplicate parts avoided.)

Your project

- What broke, and how did you fix it?
- What would you do differently with more time?
- What did the numbers tell you?

---

6. Daily Log Template

Copy this each day. It becomes your interview answers.

```
Date / Day #:
Goal today:
What I built:
What broke (and why):
How I fixed it / workaround:
Numbers measured:
Decision I made and the trade-off:
Question I still have:
```

---

7. Final Checklist (Day 10)

- [ ] Working demo: search, similar parts, Q&A with citations
- [ ] Your own modeled part included in the demo (native file + STEP)
- [ ] Can explain CAD vs. CATIA vs. AutoCAD, and what's lost converting CATIA to STEP
- [ ] Results table: success rate, time per part, storage multiplier, search quality
- [ ] Failure list with causes
- [ ] One-page architecture diagram (mini + scaled)
- [ ] "What changes at scale" notes
- [ ] 3-minute walkthrough rehearsed
- [ ] Screenshots / recording as demo backup
- [ ] Researched the job posting and the team's public AI / R&D work